Spring Boot para Eclipse

...

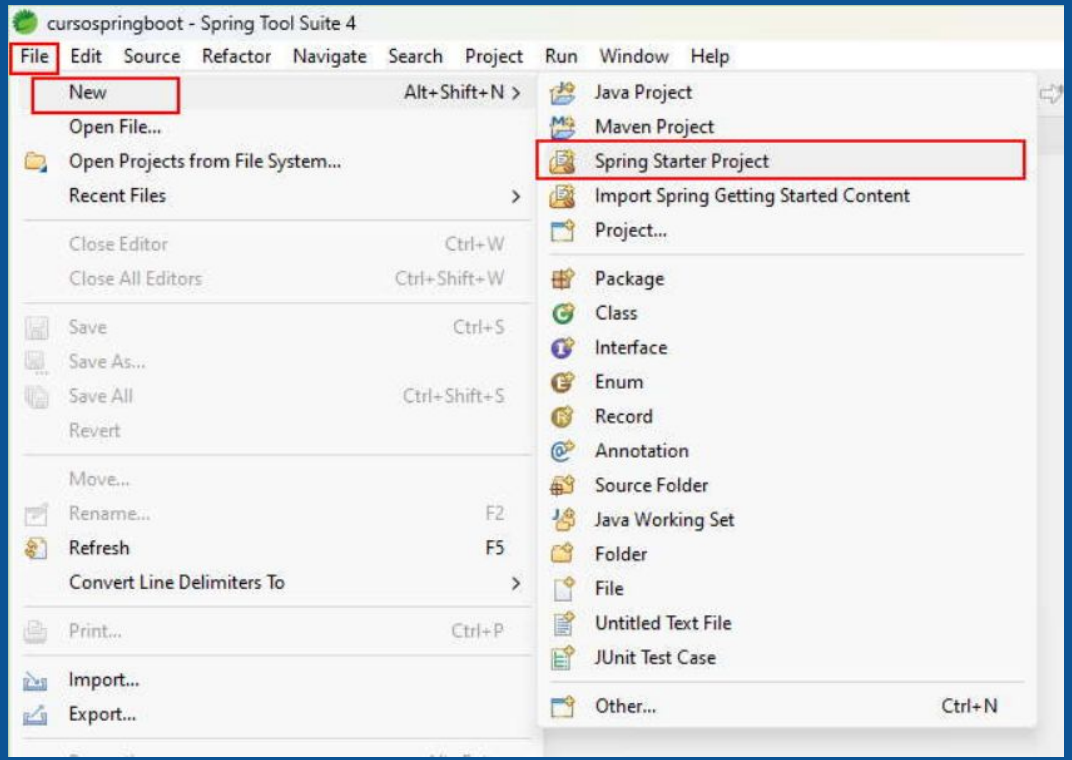
Thymeleaf y páginas dinámicas

Pasos para crear una aplicación con Spring Boot elemental que sirva páginas dinámicas

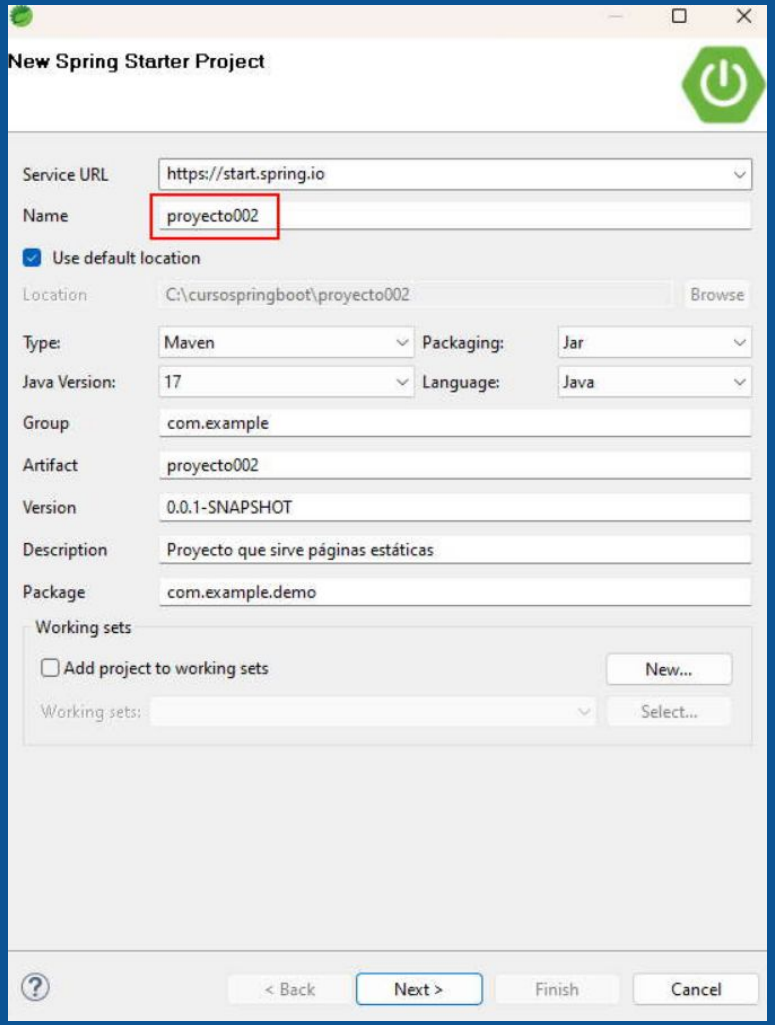
spring boot

Java

Desde el menú de opciones seleccionamos File-> New -> Spring Starter Project:

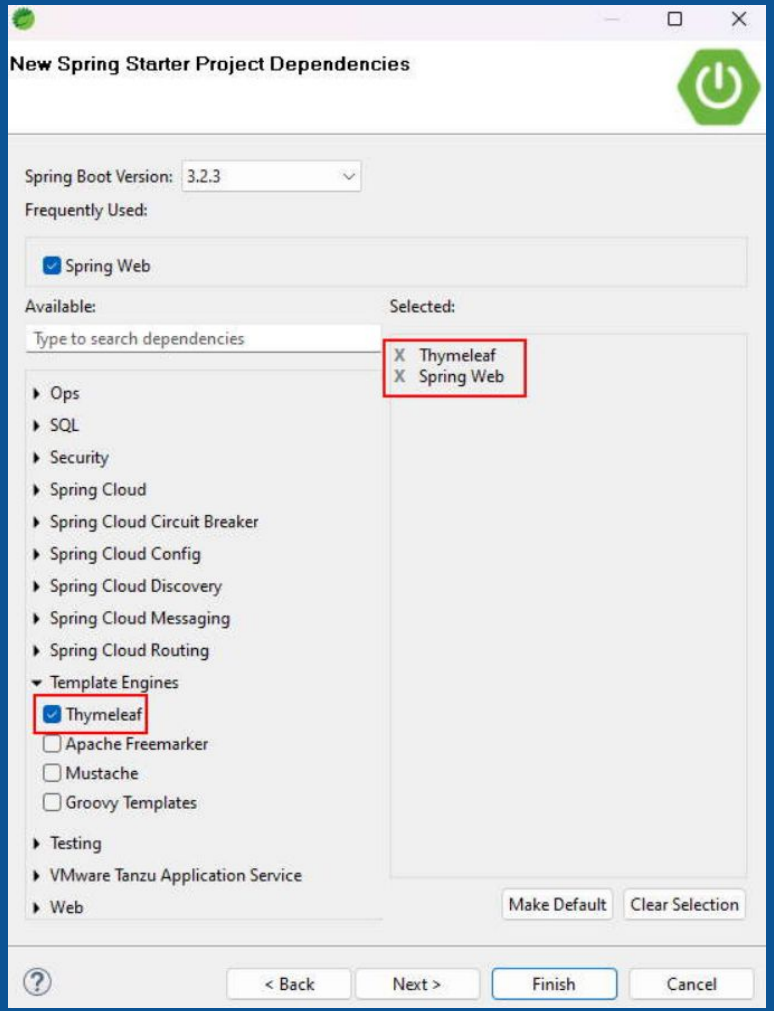


En el primer diálogo procedemos a definir el nombre de nuestro proyecto. Utilizaremos Maven como herramienta de gestión de proyecto.



spring boot

En el segundo diálogo seleccionamos ahora dos dependencias, la misma del proyecto anterior "Spring Web" y la dependencia "Thymeleaf" (en la parte superior quedan visibles las últimas dependencias usadas en proyectos anteriores, así no tenemos que buscarlas)



Java

spring boot

Java

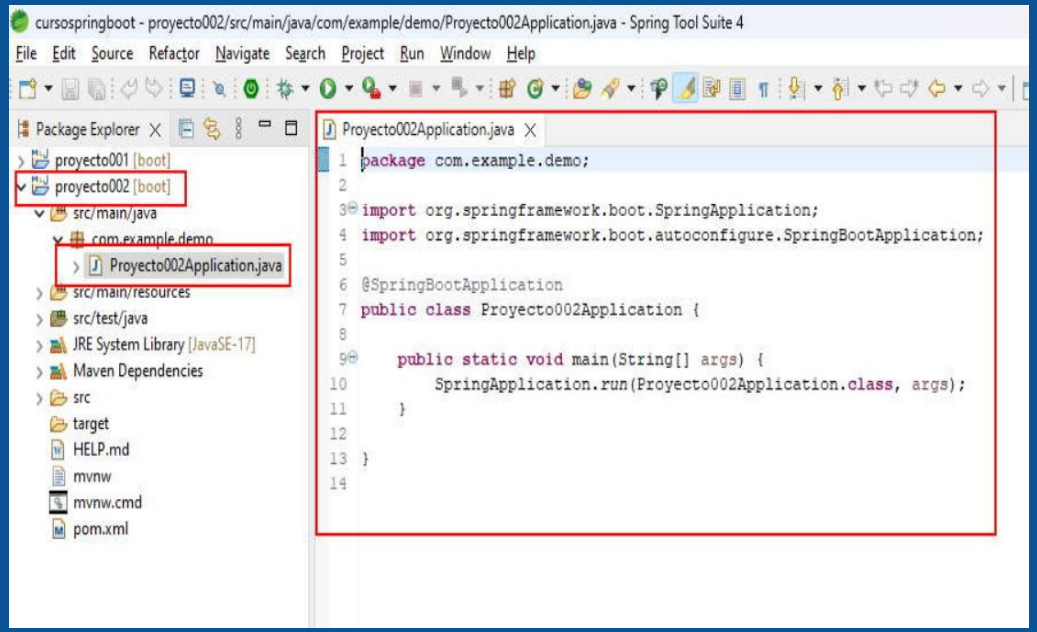
Thymeleaf es un motor de plantillas para aplicaciones web Java que se integra muy bien con el framework Spring Boot. Proporciona una manera de crear vistas HTML dinámicas fácilmente en aplicaciones Spring, permitiendo la incorporación de expresiones y atributos en el código HTML que pueden ser evaluados en el lado del servidor.

spring boot

Java

Pasemos a codificar el código Java y como se relaciona luego con la vista HTML.

Buscamos en la carpeta proyecto002\src\main\java\com\example\demo el archivo generado automáticamente llamado 'Proyecto002Application':



Esta es la clase principal de la aplicación Spring Boot, donde el método main se encuentra. Este método es el punto de entrada principal de la aplicación.

El método main tiene la siguiente llamada:

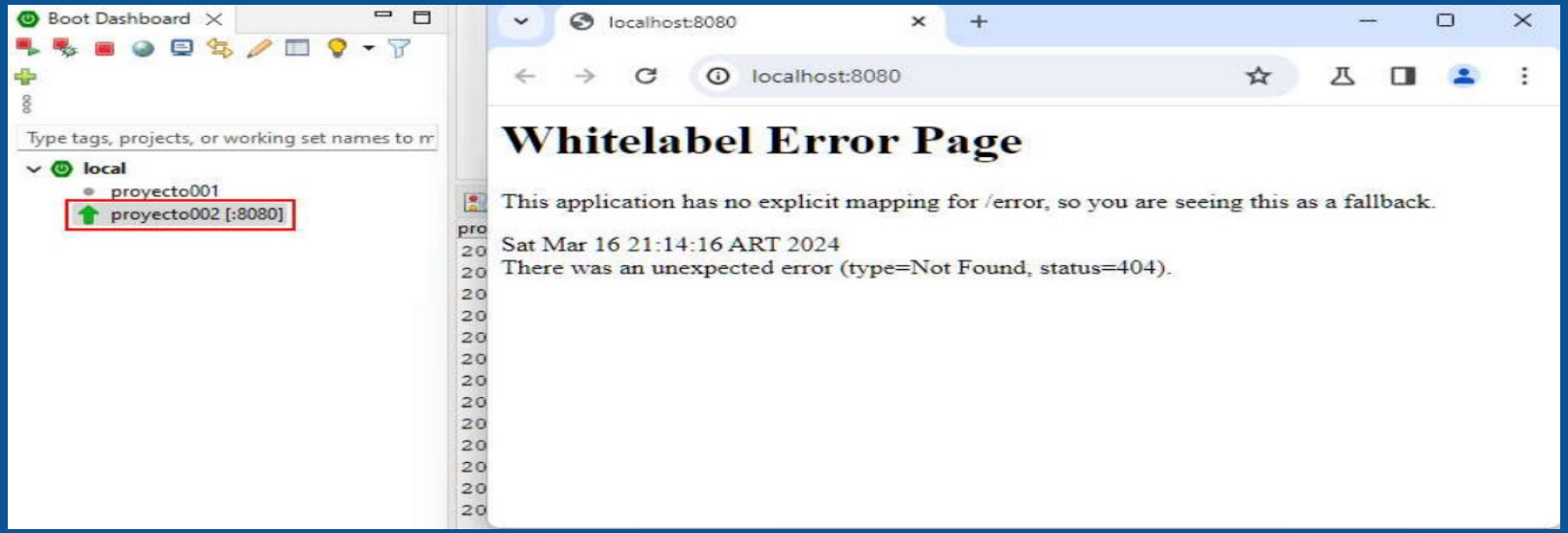
```
SpringApplication.run(Proyecto002Application.class, args);
```

El método estático 'run' inicia la aplicación Spring Boot. Toma dos argumentos: la clase principal de la aplicación (Proyecto002Application.class) y los argumentos de línea de comandos (args). Esta línea es responsable de iniciar el contenedor de Spring y desplegar la aplicación Spring Boot

spring boot

Java

Si probamos sin hacer cambios y ejecutamos como en el ejemplo anterior, podemos ver que el servidor Tomcat esta activo y nos retorna un mensaje que no hay página que pueda devolver el servidor:



spring boot

Java

Lo vamos a resolver por el momento en la forma más sencilla (luego veremos el patron MVC (Modelo Vista Controlador) para organizar nuestro código en distintas carpetas y que sea más ordenado y mantenible)

Modificamos el código del archivo con:

```java
package com.example.demo;

import java.util.ArrayList;
import java.util.List;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
@SpringBootApplication
public class Proyecto002Application {

    public static void main(String[] args) {
        SpringApplication.run(Proyecto002Application.class, args);
    }

    @GetMapping("/")
    public String inicio(Model model) {
        List<String> chistes=new ArrayList<>();
        chistes.add("¿Qué le dice un 0 a un 8? Bonito cinturón.");
        chistes.add("¿Qué hace una abeja en el gimnasio? ¡Zum-ba!");
        chistes.add("¿Cuál es el colmo de un electricista? Que su hijo sea un apagado.");
        chistes.add("¿Por qué estás hablando con la pared? ¡Porque la mesa no me responde

        String chisteAzar = chistes.get((int)(Math.random()*chistes.size()));

        model.addAttribute("chiste", chisteAzar);
        return "index";
    }
}
```

spring boot

Java

@Controller: Anota la clase como un controlador de Spring MVC.

@GetMapping("/"): Anotación que mapea la URL raíz del sitio web ("/") a este método de controlador.

```
http://www.localhost:8080/
```

spring boot

Java

Se ejecuta el método Java llamado 'inicio'.

```
model.addAttribute("chiste", chisteAzar);
```

spring boot

Java

Agrega el chiste seleccionado al modelo, que se pasará a la vista. Un poco más abajo veremos como accedemos desde la página HTML a "chiste".

```
return "index";
```

Devuelve el nombre de la vista (falta todavía que lo codifiquemos) que se debe renderizar. En este caso, la vista se llama "index".

spring boot

Java

Ahora procedamos a crear la vista en la carpeta 'template' el archivo index.html (cuidado no en la carpeta static):

cursospringboot - proyecto002/src/main/resources/templates/index.html - Spring Tool Suite 4

File Edit Source Navigate Search Project Run Window Help

Package Explorer

- proyecto001 [boot]
- proyecto002 [boot]
  - src/main/java
    - com.example.demo
      - Proyecto002Application.java
  - src/main/resources
    - static
    - templates
      - index.html
    - application.properties
  - src/test/java
  - JRE System Library [JavaSE-17]
  - Maven Dependencies
  - src
  - target
  - HELP.md
  - mvnw
  - mvnw.cmd
  - pom.xml

Proyecto002Application.java | index.html

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <meta charset="UTF-8">
6      <title>Proyecto001</title>
7  </head>
8
9  <body>
10     <h1>Chiste</h1>
11     <p th:text="${chiste}"></p>
12 </body>
13
14 </html>
```

El código fuente de la página index.html es:

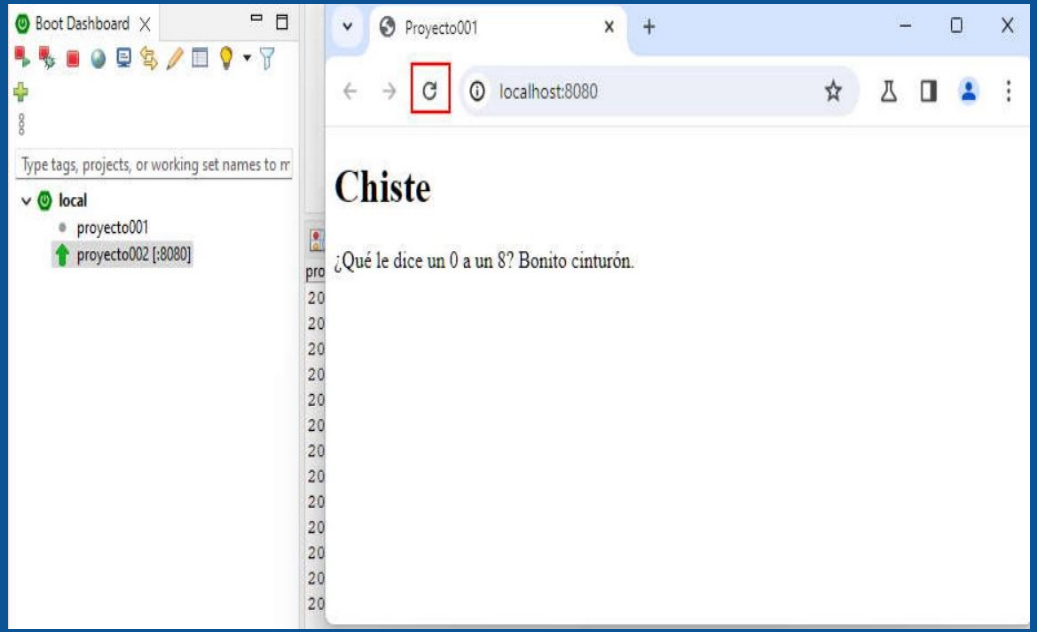
```
<!DOCTYPE html>
<html>

<head>
    <meta charset="UTF-8">
    <title>Proyecto002</title>
</head>

<body>
    <h1>Chiste</h1>
    <p th:text="${chiste}"></p>
</body>

</html>
```

th:text="${chiste}": Esta es una expresión de Thymeleaf. Thymeleaf proporciona un conjunto de atributos especiales que se pueden utilizar en elementos HTML para vincular datos del modelo de la aplicación a la vista. En este caso, th:text es un atributo especial que se utiliza para establecer el texto dentro del elemento HTML. ${chiste} es una expresión de Thymeleaf que se interpreta como una referencia al atributo chiste del modelo de la aplicación (model.addAttribute("chiste", chisteAzar);). Esto significa que el texto dentro del párrafo se establecerá dinámicamente en función del valor de la variable chiste que se pasa desde el controlador al modelo y luego a la vista.

spring boot

Java

Ahora si ejecutamos la aplicación (recordar siempre de guardar todos los archivos modificados y volver a reiniciar el servidor) vemos que nos muestra chistes distintos cada vez que actualizamos la página web:



Practica 3
Crear una pagina dinamica donde: Mediante un boton nos dirija a otra Pagina de tipo jSP